



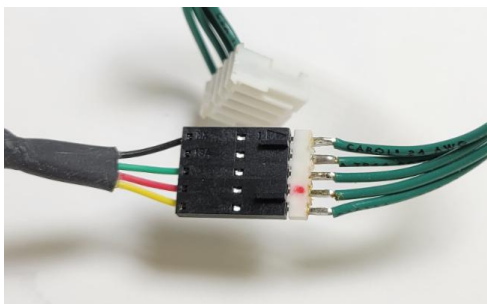
ENGS 147: Mechatronics

Lab 1: Motor Characterization

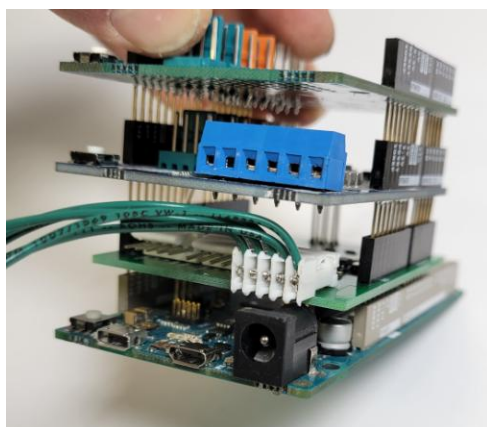
Spring 2026

Objective: In this lab, you use the Arduino stack that you assembled in Lab 0 to characterize a DC pancake motor. Modeling the motor's speed response to a voltage input as a first order system $\frac{\omega(s)}{V(s)} = \frac{K_m}{\tau s + 1}$, you will run experiments to identify values of K_m and τ under different operating conditions. First, you will connect and test each of the three Arduino shields provided in your lab kit.

1. Plug in your battery to ensure that it is charged. Leave your charger on the 1C setting.
2. Revisit Lab 0 (Arduino stack setup) to ensure that your Arduino stack is assembled correctly, that you can blink the amber LED, and that you can both program your Arduino Due and receive serial data in Serial Monitor or CoolTerm.
3. Find a pancake motor setup and record the number here: _____. You may need to work with a partner for this as there are a limited number of functional motors available for this series of labs. *Please use the same motor for each of your labs.*
4. **Encoder shield testing**
 - Connect your green encoder cable to the 5-position Molex connector coming off of the US Digital optical encoder that is attached to the top of your pancake motor. Note that the **green cable is not polarized/keyed** so you will need to align the red dot with the red cable on the encoder side.



- Connect the other end of your encoder cable to the encoder shield in your Arduino stack. Use the rightmost shield connector when viewed from the power USB port side of your Arduino Due board.



- Find the part number of the US Digital encoder on your motor. It should start with an "E". Record the encoder part number in your notes or here: _____

- Visit the US Digital website (<https://www.usdigital.com/products/encoders/>) and find the datasheet for this encoder. Record the following information:
 - Nominal supply voltage: _____
 - Counts per revolution: _____
 - Maximum shaft speed: _____
- Connect your Arduino to your laptop, open the Arduino IDE, and load the encoder shield example sketch (File > Examples > AxisEncoderShield3 > AxisEncoderShield3). Compile and upload this sketch to your Arduino Due. Open Arduino Serial Monitor or CoolTerm and connect at 9600 baud.
- Turn your motor shaft slightly in both directions. Which channel (X, Y, or Z) is being read? Which direction (CW or CCW) gives a positive encoder count? Write this on a piece of tape and attach it to your motor.
- Turn your shaft such that the output on the active channel is close to zero. Use a piece of tape to put a makeshift flag on your motor shaft, and another small piece to mark your starting position. Next, turn the shaft in the positive direction through ten revolutions and record the encoder count here: _____

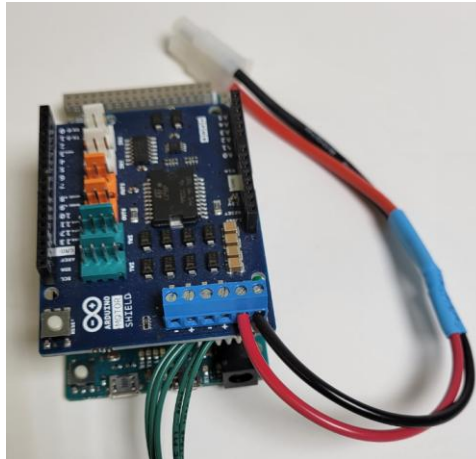


- Divide by 10. How many counts per revolution are you measuring? _____ Does this number make sense? If not, consult the datasheet for the [LS7366R-S](#) quadrature counter chips that are on the encoder shield to count pulses. Note: you can query these counters from the microcontroller on your Arduino board via a digital SPI (serial peripheral interface) bus. Consult with the instructor or a TA if you do not understand the encoder count that you measured.
- Read through the AxisEncoderShield3 Arduino sketch. Ask the instructor, a TA, or a classmate if you do not understand any of the code.
- Try changing the serial communication baud rate (between your computer and the Arduino Due) in the AxisEncoderShield3 code to test other baud rates that are supported. What is the fastest baud rate that gives you reliable output? _____
- Disconnect the Arduino from your PC and make sure the power LED goes out. You can remove the tape flag and zero point indicator that you put on your shaft and motor.

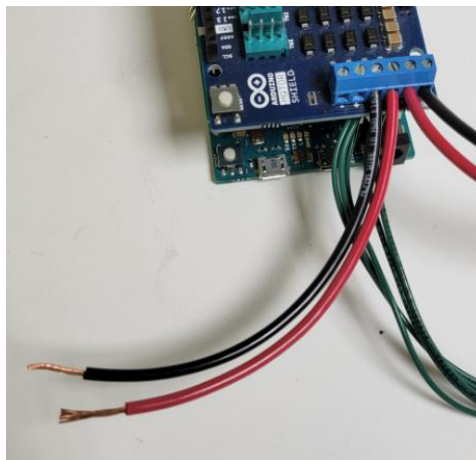
5. Motor shield testing

- Remove the 9 Axis Motion Shield from the top of your Arduino stack so that you have access to the connectors on the motor shield.

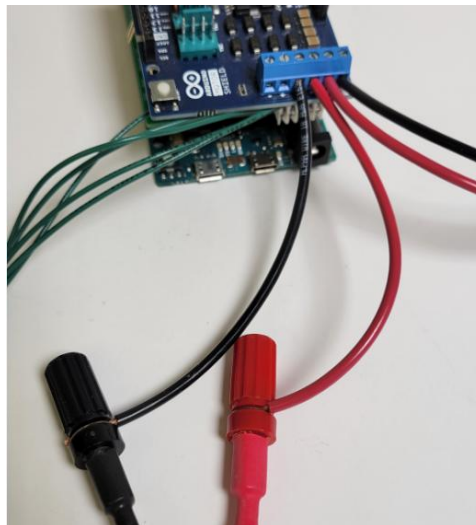
- Install your battery connector in the “Gnd” (black) and “Vin” (red) screw terminals. Ensure that the wires are pressed firmly into the connector up to their connector while tightening the screws. Once installed, you should not be able to see the copper conductors. ***Ensure that no copper strands are protruding from the connectors.***



- Install two segments of 18 gauge wire into the “+” (red) and “-” (black) terminals that are directly adjacent to the battery terminals. ***Again, ensure that the wires are fully inserted such that no copper is visible.*** You may need to trim and/or re-strip your wires.



- Connect the other end of your red and black motor wires to the red and black banana jack / screw terminals on the motor. Be sure that any exposed wires cannot touch; use electrical tape if necessary.



- Reinstall your 9 Axis Encoder Shield on the top of the Arduino stack.
- Connect your 9.6V 2000mAh NiMH battery to the battery connector.
- Connect your laptop to the Arduino.
- Open the Arduino IDE and load the motor shield example sketch (File > Examples > ArduinoMotorShield3 > Demo). Read through this code and make sure you understand what it will do. When you are ready, compile and upload the sketch to your Duo.
- Your motor should start running. Switch to Serial Monitor or CoolTerm and ensure that you can see serial output from the motor demo program.
- Which way does the motor turn when it is given a positive voltage (i.e. commanded with a positive PWM value)? Is this consistent with the positive direction of the encoder?
- **Important:** *Always unplug the USB cable from your laptop before unplugging the NiMH battery. When battery power is not available, the Arduino motor shield will attempt to drive the motor using power sourced from your laptop USB port. Your laptop should limit the supplied current, but unplug to be safe.*

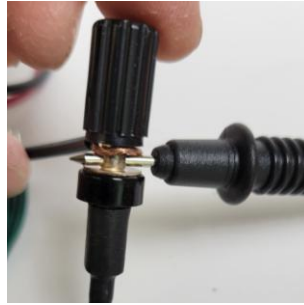
6. Motion shield (IMU) testing

- Open the Arduino IDE and load the 9 Axis Motion Shield demo sketch (File > Examples > AxisMotionShield9 > AxisMotionShield9). Compile and upload this sketch to your Arduino Due board.
- Open Serial Monitor or CoolTerm and view the output. All values reported by AxisMotionShield9 are linear accelerations; however, the motion shield also contains rate gyroscopes and magnetometers. Can you verify the orientation of the coordinate frame printed on the motion shield? You may wish to pause the display by unclicking the double downward chevron icon (Arduino Serial Monitor) or dragging the scrollbar upward (CoolTerm).
- Note: you will not need the motion shield for the lab assignments but may wish to use it for your project. You can view the relevant documentation here:
 - Arduino page: <https://store-usa.arduino.cc/products/arduino-9-axis-motion-shield>
 - Bosch BNO055 documentation: <https://www.bosch-sensortec.com/products/smart-sensor-systems/bno055/#documents>

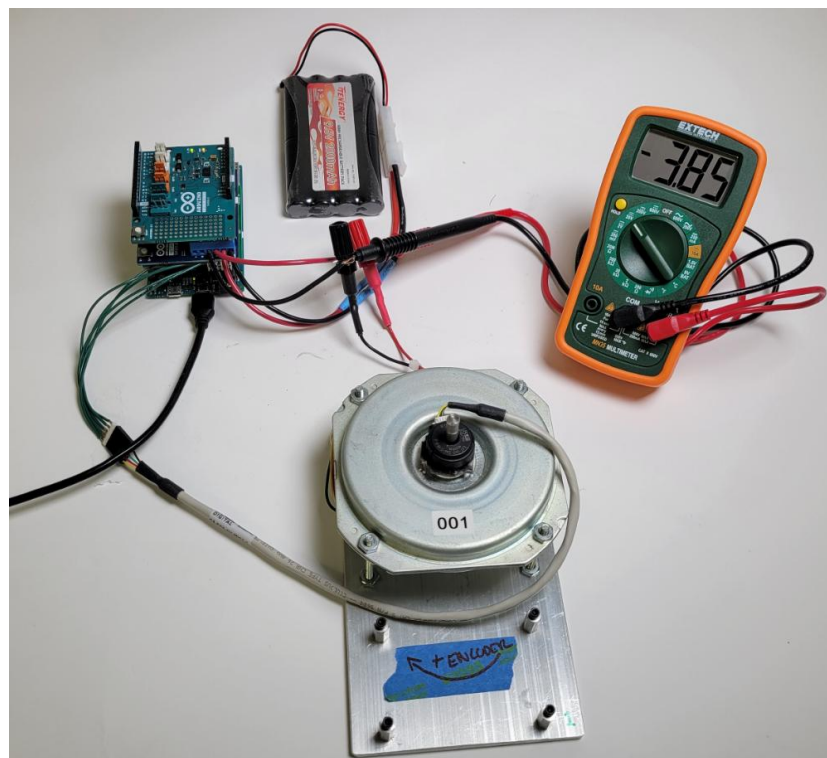
7. Motor characterization

- In this portion of the lab, you will write your own code to capture speed data from the encoder while the motor responds to a variety of step inputs. You will generate a separate data file for each step input, which you will process in MATLAB or Excel to compute K_m and τ .
- Locate your handheld multimeter, turn it on, and ensure that the low battery indicator is **not** displayed. If your multimeter battery is low, replace it now. The instructor or a TA can help if necessary. Extra batteries are available in the tool bins in Couch I and from the Instrument Room.

- Attach the leads from your multimeter to the positive and negative battery terminals. There are many ways to do this. **You may wish to use alligator clips to connect your multimeter probes**, or unscrew the terminals, loop the wire around the terminal, insert the probe tip through the terminal cross-hole, and tighten (see image below). *Be very careful not to short the motor terminals together as this will cause your battery to overheat when you attempt to run the motor.*



- Your complete setup should look like this:



- Using the encoder shield and motor shield example sketches as a guide, write a new Arduino sketch that does the following:
 - In the `setup()` routine:
 - Initialize the serial connection
 - Initialize the motor shield
 - Initialize the encoder shield to read one channel
 - Include a delay (e.g. `delay(200)`) to ensure all configuration is complete
 - In the `loop()` routine (note: we will **NOT** actually use this as a loop!):
 - Declare and initialize variables including storage array(s)
 - Set motor PWM to zero, wait 1sec, then set motor to a desired PWM value

- Enter a timed loop that **repeats every 5ms** for a total duration of ten seconds
 - Suggestion: start from the example at <https://dartgo.org/147examples>. First try simply enforcing loop timing at 5ms for one second, then print out a list of the absolute times in microseconds that your code enters the TS if() statement. You'll need to be sure you know how to store values in an array.
 - Extend to perform time-critical operations (e.g. read encoder, compute speed) inside the TS if() statement. You should compute speed in rad/s.
 - Perform operations that are not time critical (e.g. store time, position, and speed)
- Once loop exits after ten seconds:
 - Set motor PWM to zero
 - Print data via serial port in CSV format (e.g. timestamp, position, speed in rad/s)
- Once your program is working, you will need to fill in the table below. For each row:
 - Update your Arduino sketch to set the motor step input to the prescribed PWM value
 - Compile your sketch and upload to the Arduino
 - Capture data in Serial Monitor via copy/paste (make sure you clear your screen at an appropriate time!) or CoolTerm by selecting Connection > Capture to Text/Binary File > Start
 - Once the motor is running at steady state, record the voltage displayed on your multimeter
 - Stop recording with Connection > Capture to Text/Binary File > Stop
 - Import data into MATLAB or Excel and compute K_m (with two different sets of units) and τ

You may automate some/all of this process if you find that helpful!

PWM Command	Voltage [V]	K_m [rad/s / pwm]	K_m [rad/s / V]	τ [s]
-400				
-350				
-300				
-250				
-200				
-150				
-100				
-75				
-50				
-25				
25				
50				
75				
100				
150				
200				
250				
300				
350				
400				

- Include the following information in your lab writeup, and be prepared to demonstrate and discuss it during your lab checkoff:
 - (20 pts) Your completed data table. Note that K_m should be positive. If you get negative values consider adding a negative sign to your speed calculation to account for encoder mounting.
 - (20 pts) Your Arduino sketch with timed loop implemented as covered in lecture.
 - (20 pts) A time-series plot from one of your data captures, annotated to illustrate how you extracted K_m and τ . Provide code if you wrote a script in MATLAB, Python, etc. to do this.
 - (20 pts) Produce the following three plots (you may wish to collect additional data in any regions that appear nonlinear):
 - Motor speed (y axis) vs. voltage across motor (x axis)
 - Motor speed (y axis) vs. PWM command (x axis)
 - Voltage across motor (y axis) vs. PWM command (x axis)
 - (20 pts) Discussion of:
 - Likely sources of error in your data
 - How you would use your results to implement closed-loop motor speed control